

UNDERSTAND BASIC CONCEPT OF C++ PROGRAMMING LANGUAGE

1.1 WHAT DOES C PLUS PLUS PROGRAMMING LANGUAGE (C++) MEAN?

C++ (said C plus plus) is an object-oriented computer language created by notable computer scientist Bjarne Stroustrup as part of the evolution of the C family of languages. It was developed as a cross-platform improvement of C to provide developers with a higher degree of control over memory and system resources.

Some call C++ “C with classes” because it introduces object-oriented programming principles, including the use of defined classes, to the C programming language framework. Over time, C++ has remained a very useful language not only in computer programming itself, but in teaching new programmers about how object-oriented programming works. However, it does not support only object-oriented, but also procedural and functional. Thanks to its high flexibility and scalability, C++ can be used to develop a broad range of software, applications, browsers, Graphical User Interfaces (GUIs), operating systems, and games.

Today C++ is still very appreciated for its notable portability which allows developers to create programs that can run on different operating systems or platforms very easily. Despite being a high-level language, since C++ is still close to C it can be used for low-level manipulation due to its close relation with machine language.

C++ is a **general-purpose programming language** created by Bjarne Stroustrup as an extension of the C programming language, or "C with Classes ". The language has expanded significantly over time, and modern C++ now has object-oriented, generic, and functional features in addition to facilities for low-level memory manipulation.

Designed by: Bjarne Stroustrup

Developer: ISO/IEC JTC 1 (Joint Technical Committee 1) / SC 22 (Subcommittee 22) / WG 21 (Working Group 21)

Family: C

Paradigms: Multi-paradigm: procedural, imperative, functional, object oriented, generic, modular

Reference: en.wikipedia.org/wiki/C%2B%2B

1.2 EXPLAIN THE GENERAL FEATURES OF C++

Features of C++

C++ is an upgraded version of C programming. The main idea behind creating C++ programming was to add object orientation to the C programming language. The major up gradations are object-oriented programming methodology, namespace feature, operator overloading, error & exception handling. The motivation behind object-oriented programming is to try to see the whole world in the form of classes & objects.

There are various features of C++ such as,

- Object Oriented
- Simple
- Platform Dependent
- Mid-level programming language
- Structured programming language
- Rich Library
- Memory Management
- Powerful & Fast
- Pointers
- Compiler based
- Syntax based language

Object Oriented Programming language

The main up-gradation from C to C++ is object-oriented programming. It follows concept of oops like polymorphism, inheritance, encapsulation, abstraction. This makes development and maintenance easier. Let's briefly understand the concepts of object-oriented programming.

- **Class:** A class is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type.
- **Object:** It is a basic unit of Object-Oriented Programming and represents the real-life entities. A C++ program creates many objects which interact by invoking methods.
- **Polymorphism:** Polymorphism refers to the ability of OOPs programming languages to differentiate between entities with the same name efficiently.
- **Inheritance:** Inheritance is the mechanism in which one class is allowed to inherit the features (fields and methods) of another class.
- **Encapsulation:** Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates.
- **Abstraction:** Data Abstraction is the property by virtue of which only the essential details are displayed to the user. The trivial or the non-essentials units are not displayed to the user.

Features of C++:

Simple: C++ provides a structured approach wherein you can break the problem into parts and design the solution modularly. It provides you a rich set of library functions that you can use while implementing the solution. If you have worked with **C language**, then moving to C++ would be a very smooth transitioning. The syntax is almost similar with minute changes.

Platform Dependent: Platform dependent language means the language in which programs can be executed only on that operating system where it is developed & compiled. It cannot run or execute it on any other operating system.

C++ is a platform-dependent language. Having said that, C++ programs can be executed in many machines with little bit or no change.

Mid-level programming language: C++ has the ability to do both low-level & high-level programming. This is the reason why C++ is known as a mid-level programming language. When we talk about low-level programming, C++ is used to develop system applications such as the kernel, driver, etc.

Structured programming language: In C++ programming, the code is modular with the help of functions, classes & objects, and the modules are loosely coupled. Modular code is easy to understand & modify. This makes C++ a structured programming language.

Rich Library: Developers have access to lots of in-built functions provided by C++ language. This saves time & makes development fast. Let's look at some of the C++ header files & functionalities provided by it.

Memory Management: C++ supports dynamic memory allocation. You can free the allocated memory at any time. Not only this C++ also provides dynamic memory management techniques.

Powerful & Fast: C++ is a fast language as compilation and execution time is less. Also, it has a wide variety of data types, functions & operators.

Pointers: are variables that store the address of another variable. Pointer points to the memory location of a variable. C++ supports pointer and provides solutions to lots of problems that demand access to memory location.

Compiler based: C++ is a compiler-based programming language. Without compilation, no C++ program can be executed. The compiler first compiles the C++ program and then it is executed.

C++ Features: Syntax based language: C++ is a language that complies strongly with syntax. Language following rules and regulations very strictly is known as tight syntax-based language. C, C++, Java, .net are some of the examples.

1.3 WHY IS C++ OBJECT-ORIENTED PROGRAMMING LANGUAGE?

Why C++ is Object Oriented Programming Language?

Any programming language that makes use of objects, classes, added to its abstraction, encapsulation, and polymorphism are said to be **Object-Oriented Programming Language** or OOPs.

C++ also makes use of these concepts, but some of the above concepts are not fully optimized by C++ therefore it is sometimes also called partial Object-Oriented Programming Language.

Every programming language starts execution from the main class but in C++ the main function that starts initiates the execution is outside the main class.

This is java class execution.

```
class hello
{
    public static void main(String args[])
    {
        System.out.println("Hello World");
    }
}
```

In this, the execution of code begins with the main class that is hello.

But in C++ we can write and execute a fully functional code even without declaring a single class because in C++ the execution begins from the main class which comes by default and outside any class.

This is C++ execution.

```
using namespace std;

int main()
{
    cout << "Hello World";
    return 0;
}
```

In this, the execution begins from the main class, and note that there is no class declared in this code even then C++ is capable of giving output.

This is the reason why C++ is also called partially object-oriented programming.

1.4 EXPLAIN THE IMPORTANCE OF C++

Following are the advantages of C++ –

- C++ is a highly portable language and is often the language of selection for multi-device, multi-platform app development.
- C++ is an object-oriented programming language and includes concepts like classes, inheritance, polymorphism, data abstraction, and encapsulation which allow code reusability and makes programs very maintainable.
- C++ use multi-paradigm programming. The Paradigm means the style of programming .paradigm concerned about logics, structure, and procedure of the program. C++ is multi-paradigm means it follows three paradigms Generic, Imperative, Object Oriented.
- It is useful for the low-level programming language and very efficient for general purpose.
- C++ gives the user complete control over memory management. This can be seen both as an advantage and a disadvantage as this increases the responsibility of the user to manage memory rather than it being managed by the Garbage collector.
- The wide range of applications – From GUI applications to 3D graphics for games to real-time mathematical simulations, C++ is everywhere.
- C++ has a huge community around it. Community size is important, because the larger a programming language community is, the more support you would be likely to get. C++ is the 6th most used and followed tag on Stack Overflow and GitHub.
- C++ has a very big job market as it is used in various industries like finance, app development, game development, Virtual reality, etc.
- C++'s greatest strength is how scalable it could be, so apps that are very resource intensive are usually built with it. As a statically written language, C++ is usually more performant than the dynamically written languages because the code is type-checked before it is executed.
- Compatibility with C – C++ is compatible with C and virtually every valid C program is a valid C++ program.

UNDERSTAND THE FUNDAMENTAL DATA TYPES, CONSTANTS AND VARIABLES OF C++

2.1 EXPLAIN DATA TYPES AND VARIABLES

All variables use data-type during declaration to restrict the type of data to be stored. Therefore, we can say that:

DATA TYPES are used to tell the variables the type of data it can store. Whenever a variable is defined in C++, the compiler allocates some memory for that variable based on the data type with which it is declared. Every data type requires a different amount of memory.

C++ supports a wide variety of data types and the programmer can select the data type appropriate to the needs of the application. Data types specify the size and types of value to be stored. However, storage representation and machine instructions to manipulate each data type differ from machine to machine, although C++ instructions are identical on all machines.

C++ supports the following data types & is mainly divided into three types:

- ❖ Primary or Built in or Fundamental data type
- ❖ Derived data types
- ❖ User defined data types

Primitive Data Types: These data types are built-in or predefined data types and can be used directly by the user to declare variables. example: int, char, float, bool, etc. Primitive data types available in C++ are:

- Integer
- Character
- Boolean
- Floating Point
- Double Floating Point
- Valueless or Void
- Wide Character

Derived Data Types: The data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types. These can be of four types namely:

- Function
- Array
- Pointer
- Reference

Abstract or User-Defined Data Types: These data types are defined by the user itself. Like, as defining a class in C++ or a structure. C++ provides the following user-defined datatypes:

- Class
- Structure
- Union
- Enumeration

VARIABLE: It is the name for memory location that may be used to store a data value. A variable may take different values at different times during execution. A variable name may be chosen by the programmer in a meaningful way, so as to reflect its function (or) nature in the program. For example, sum, avg, total etc.

The rules for naming a variable are explained below-

- ❖ They must begin with a letter.

- ❖ Maximum length of variable is 31 characters in ANSI standard. But, first eight characters are significant by many compilers.
- ❖ Upper and lowercase characters are different. For example: total, TOTAL, Total are 3 different variables.
- ❖ The variable is not to be a keyword.
- ❖ White space is not allowed.

Variable declaration

The syntax and example with regards to variable declaration are explained below –

Syntax

Given below is the syntax for variable declaration –

Datatype v1,v2,... vn;

Where, v1, v2,...vn are names of variables.

For example,

int sum;

float a,b;

Variable can be declared in two ways –

- **Local declaration** – ‘Local declaration’ is declaring a variable inside the main block and its value is available within that block.
- **Global declaration** – ‘Global declaration’ is declaring a variable outside the main block and its value is available throughout the program.

Example

Following is the C program for local and global declaration of variables in C language –

```
int a, b; /* global declaration*/
main () {
    int c; /* local declaration*/
    - - -
}
```

2.2 LIST VARIOUS TYPES OF OPERATION

An operator is a symbol used for performing operations on operands. An operator operates operands. The operations can be mathematical or logical. There are different types of operators in C++ for performing different operations.

Consider the following operation:

a = x + y;

In the above statement, x and y are the operands while + is an addition operator. When the C++ compiler encounters the above statement, it will add x and y and store the result in variable a.

Types of Operators in C++

There are mainly 6 different types of operators in C++

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Other Operators

Arithmetic Operators: They are the types of operators used for performing mathematical/arithmetic operations. They include: +, -, X, etc.

Relational Operators: These types of operators perform comparisons on operands. For example, you may need to know which operand is greater than the other, or less than the other. They include: =, <=, >=, etc

Logical Operators: combine two/more constraints/conditions. Logical operators also complement evaluation of original condition under consideration. They include: && (AND) logical conjunction of expression, || (OR) logical disjunction of expressions,

Bitwise operators: perform bit-level operations on operands. First, operators are converted to bit level then operations are performed on the operands. When arithmetic operations like addition and subtraction are done at bit level, results can be achieved faster. They include: Bitwise AND: converts the two operands into binary and performs conjunctive operation bit by bit. Bitwise OR: converts the two operands into binary and performs disjunctive operation bit by bit.

Assignment operators: assign values to variables. The operand/variable is added to left side of the operator while the value is added to the right side of the operator. The variable and the value must belong to the same data type, otherwise, the C++ compiler will raise error. =, = +, = -,

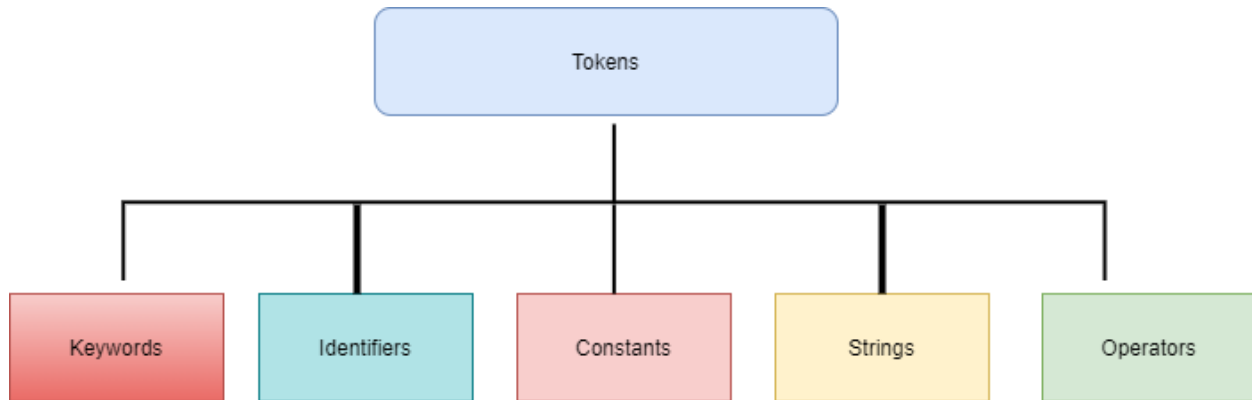
Other Operators: include size of operator, Comma Operator, Conditional Operator, and Operators Precedence. Let us discuss other operators supported by C++:

2.3 DEFINE KEYWORDS, IDENTIFIER, CONSTANTS, ESCAPE SEQUENCE ETC.

C++ is a powerful language. In C++ we can write structured programs and object-oriented programs also. C++ is a superset of C and therefore most constructs of C are legal in C++ with their meaning unchanged. However, there are some exceptions and additions.

Token: When the compiler is processing the source code of a C++ program, each group of characters separated by white space is called a token. Tokens are the smallest individual units in a program. A C++ program is written using tokens. It has the following tokens:

- Keywords
- Identifiers
- Constants
- Strings
- Operators



Keywords (also known as **reserved words**) have special meaning to the C++ compiler and are always written or typed in short (lower) cases. Keywords are words that the language uses for a special purpose, such as **void**, **int**, **public**, etc. It can't be used for a variable name or function name. Below is the table for the complete set of C++ keywords.

Identifiers refer to the name of variables, functions, arrays, classes, etc. created by the programmer. They are the fundamental requirement of any language.

Rules for naming identifiers:

- Identifier name cannot start with a digit or any special character.
- A keyword cannot be used as an identifier name.
- Only alphabetic characters, digits, and underscores are permitted.
- The upper case and lower case letters are distinct. i.e., A and a are different in C++.
- The valid identifiers are GFG, gfg, geeks-for-geeks.

Different between keywords & identifiers

So there are some main properties of keywords that distinguish keywords from identifiers:

- Keywords are predefined/reserved words and identifiers are the values used to define different programming items like a variable, integers, structures, unions.
- Keywords always start with lowercase whereas identifier can start with the uppercase letter as well as a lowercase letter.
- A keyword contains only alphabetical characters, but an identifier can consist of alphabetical characters, digits, and underscores.
- No special symbol, punctuations used in keywords and identifiers. The only underscore can be used in an identifier.
- Example of keywords and identifiers:

- **Keywords:** int, char, while, do.
- **Identifiers:** Geeks-for-Geeks, GFG, Gfg1.

Constants: As the name suggests the name constants are given to such variables or values in C/C++ programming language which cannot be modified once they are defined. They are fixed values in a program. There can be any types of constants like integer, float, octal, hexadecimal, character constants, etc. Every constant has some range. The integers that are too big to fit into an int will be taken as long. Now there are various ranges that differ from unsigned to signed bits. Under the signed bit, the range of an int varies from -128 to +127, and under the unsigned bit, int varies from 0 to 255.

Escape sequences are a sequence of characters that represent a different meaning when used in a string literal. Escape sequences are the same for many other programming languages like *java*, *c++*, *c#*, etc. They usually begin with the ‘\ (backslash)’ symbol.

Format specifiers: basically help us work with different types of data types. Format specifiers are generally used during standard input and standard output procedures in C programming Language also referred to as *formatted input* and formatted output. C does not allow the user to print or input the values straightforward. The programmer must make use of the format specifiers.

INPUT AND OUTPUT OPERATIONS IN C++

3.1 IDENTIFY INPUT / OUTPUT OPERATORS IN C++

INPUT OPERATOR: commonly known as the extraction operator (>>), is used with the standard input stream, cin. As stated earlier, cin treats data as a stream of characters. These characters flow from cin to the program through the input operator. The input operator works on

two operands, namely, the c in stream on its left and a variable on its right. Thus, the input operator takes (extracts) the value through cin and stores it in the variable. To understand the concept of an input operator, consider this example.

A program to demonstrate the working of an input operator.

```
#include
using namespace std;
int main () {
    int a;
    cin>>a;
    a = a+1;
    return 0;
}
```

In this example, the statement cin>> a takes an input from the user and stores it in the variable a.

OUTPUT OPERATOR: commonly known as the insertion operator (<<), is used. The standard output stream cout like cin, cout also treats data as a stream of characters. These characters flow from the program to cout through the output operator. The output operator works on two operands, namely, the cout stream on its left and the expression to be displayed on its right. The output operator directs (inserts) the value to cout.

A program to demonstrate the working of an output operator.

```
#include
using namespace std;
int main () {
    int a;
    cin>>a;
    a=a+1;
    cout<<a;
    return 0;
}
```

This example is similar to Example 1. The only difference is that the value of the variable a is displayed through the instruction cout << a.

3.2 DISCOVER INPUT /OUTPUT WITH STREAMS

The ‘cin’ C++ Stream Object: The word **cin** stands for console input. It is pronounced as “see in”. it is the standard input C++ **stream** object. It is used as an input statement to get input from user through keyword during the execution of the program. This C++ **stream** object represents the flow of data from input device (i.e. keyboard) to the computer memory. The cin **stream** input object is also part of iostream.h header file.

The ‘cout’ C++ Stream Object: The cout is pronounced as See Out. The cout stands for console output. The cout is a c++ standard output C++ **stream** object. It is used as an output statement to display. Output on the computer screen. This C++ **stream** object represents the flow of data from computer memory to the display screen. It is a part of iostream.h header file.

3.3 DESCRIBE THE CONCEPT OF OBJECT LIFE TIME

Object lifetime is a useful concept when reasoning about program semantics and correctness. When the lifetime of an object has not yet begun or has already ended, there is no object. It may be that sub objects exist, e.g. during the execution of constructors and destructors, but the object in question itself does not exist. If there is no object, it can have no state, and no invariants can be met.

That means that we have to be careful when we call member functions in a constructor or destructor, because that member function may rely on an invariant that has not yet been established or has been destroyed already. It also means that the cleanup we perform in a destructor should not be able to cause too much trouble: We can’t fix a broken object that does not exist anymore.

UNDERSTAND FUNCTIONS AND LIBRARIES IN C++

4.1 DEFINE FUNCTION

A FUNCTION: is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. Functions are used to perform certain actions, and they are

important for reusing code: Define the code once, and use it many times. Types of function are *Built-in Functions/library Functions & User-defined Functions*.

Significances

- Use of functions enhances the *readability* of a program. A big code is always difficult to read. Breaking the code in smaller Functions keeps the program organized, easy to understand and makes it reusable.
- The C compiler follows *top-to-down* execution, so the control flow can be *easily managed* in case of functions. The control will always come back to the *main()* function.
- It reduces the complexity of a program and gives it a *modular structure*.
- In case we need to test only a particular part of the program we will have to run the whole program and figure out the errors which can be quite a complex process. Another advantage here is that functions can be *individually tested* which is more convenient than the above mentioned process.
- A function can be used to create our own *header file* which can be used in any number of programs i.e. the reusability.

4.2 EXPLAIN INLINE, OVERLOADED AND RECURSIVE FUNCTIONS

INLINE FUNCTION: By using function the size of the program is reduced. It is because a defined function can be called at multiple places in a program by using only a single call statement. When the compiler compiles the program, it generates a special code at the function call to jump to the function definition. It also generates another special code in the function definition (usually at the end of a function definition) to transfer back the control to the calling function. Thus, extra execution time is spent during the execution of the program to jump the control from the calling function to function definition and then return back to the calling function.

FUNCTION OVERLOADING: allows us to create multiple functions with the same name, so long as each identically named function has different parameter types (or the functions can be otherwise differentiated). Each function sharing a name (in the same scope) is called an **overloaded function** (sometimes called an **overload** for short).

RECURSION: is a method in C++ which calls itself directly or indirectly until a suitable condition is met. In this method, we repeatedly call the function within the same function, and it has a base case and a recursive condition. The recursive condition helps in the repetition of code again and again, and the base case helps in the termination of the condition.

4.3 EXPLAIN PASSING AND DEFAULT ARGUMENTS

PASSING ARGUMENTS: in C and C++ language are copied to the program stack at run time, where they are read by the function. These arguments can either be values in their own right, or they can be pointers to areas of memory that contain the data being passed. Passing a pointer is also known as passing a value **by reference**.

DEFAULT ARGUMENT: is a value provided in a function declaration that is automatically assigned by the compiler if the calling function doesn't provide a value for the argument. In case any value is passed, the default value is overridden.

4. 4 DIFFERENTIATE LIBRARY FUNCTIONS AND USES

Library / built-in functions	User-defined functions
These functions are predefined in a header file or preprocessor directive.	These functions are not predefined rather it is defined by user according to the requirements.
These functions can be simply used by including respective header file.	These functions should be declared, defined and called in order to use.
Since, these functions are predefined programs are short.	Since, these functions should be defined by the user programs are lengthy.
Program execution time is faster.	Program execution time is slower.
It simplifies the program.	Using more user-functions increase complexity.
Eg, strlen(), strcmp(), strcpy(), strcat()	Eg, fact(), average(), greatest(), anyone()

UNDERSTAND OBJECT ORIENTED PROGRAMMING CONCEPTS

5.1 DEFINE CLASSES

CLASS: A class in C++ is the building block that leads to Object-Oriented programming. It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A C++ class is like a blueprint for an object. For Example: Consider the Class of Cars. There may be many cars with different names and brand but all of them will share some common properties like all of them will have 4 *wheels*, *Speed Limit*, *Mileage range* etc. So here, Car is the class and wheels, speed limits, mileage are their properties.

5.2 EXPLAIN CONCEPT OF METHODS

Concept and Definition: Method study is basically conducted to simplify the work or working methods and must go towards higher productivity. It is always desirable to perform the requisite function with desired goal minimum consumption of resources. Method signifies how a work is to be done i.e. description of how we consume resources *in order to achieve our target?* “A procedure for examining the various activities associated with the problem which ensures a systematic, objective and critical evaluation of the existing factors and in addition and imaginative approach while developing improvements”.

5.3 DEFINE OBJECTS

AN OBJECT: is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

5.4 DEFINE CONSTRUCTOR METHODS

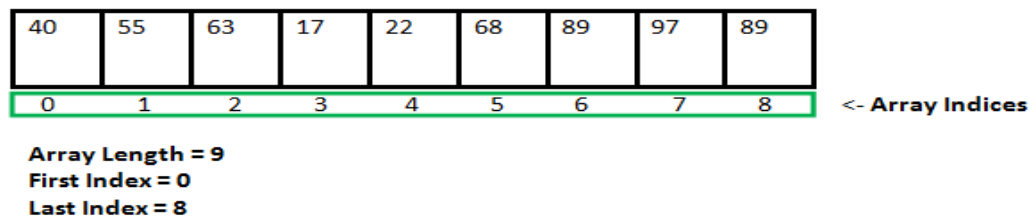
CONSTRUCTOR IN C++: is a special method that is invoked automatically at the time of object creation. It is used to initialize the data members of new objects generally. The constructor in C++ has the same name as the class or structure. Constructor is invoked at the time of object creation. It constructs the values i.e. provides data for the object which is why it is known as constructors.

UNDERSTAND ARRAYS AND POINTERS

6.1 DEFINE AND INITIALIZE ARRAYS

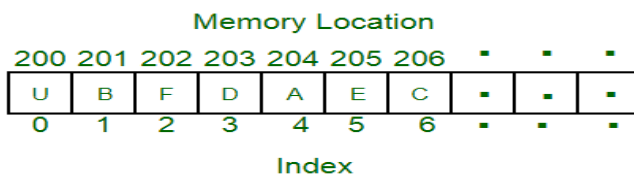
AN ARRAY in C/C++ or be it in any programming language is a collection of similar data items stored at contiguous memory locations and elements can be accessed randomly using

indices of an array. They can be used to store the collection of primitive data types such as int, float, double, char, etc of any particular type. To add to it, an array in C/C++ can store derived data types such as structures, pointers etc. Given below is the picture representation of an array.



6.2 EXPLAIN ONE DIMENSIONAL ARRAYS

- It is a list of the variable of similar data types.
- It allows random access and all the elements can be accessed with the help of their index.
- The size of the array is fixed.
- For a dynamically sized array, vector can be used in C++.
- Representation of 1D array:



6.4 Explain Multidimensional Array

- It is a list of lists of the variable of the same data type.
- It also allows random access and all the elements can be accessed with the help of their index.
- It can also be seen as a collection of 1D arrays. It is also known as the Matrix.
- Its dimension can be increased from 2 to 3 and 4 so on.
- They all are referred to as a multi-dimension array.
- The most common multidimensional array is a 2D array.
- Representation of 2 D array:

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

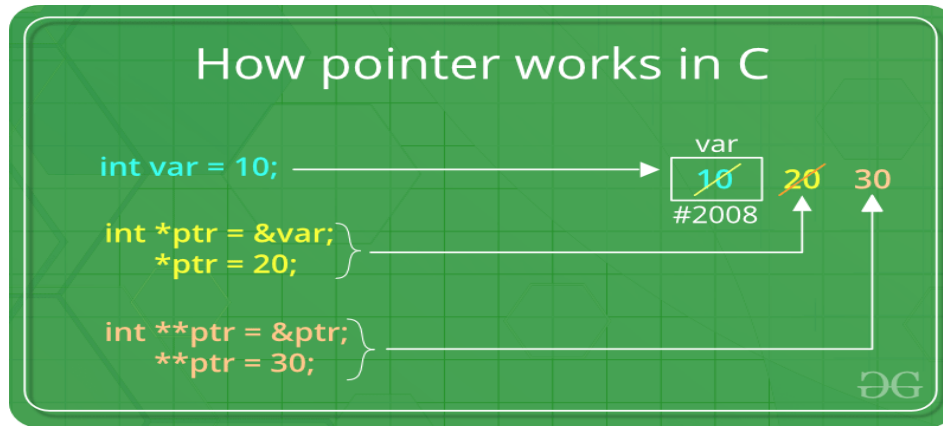
6.3 DEFINE POINTER

POINTERS: are symbolic representations of addresses. They enable programs to simulate call-by-reference as well as to create and manipulate dynamic data structures. It's general declaration in C/C++ has the format:

Syntax:

datatype *var_name;


```
int *ptr; //ptr can point to an address which holds int data
```



6.4 EXPLAIN POINTER ARITHMETIC

A pointer in c is an address, which is a numeric value. Therefore, you can perform arithmetic operations on a pointer just as you can on a numeric value. There are four arithmetic operators that can be used on pointers: ++, --, +, and -

6.5 EXPLAIN ARRAYS OF POINTER

Array name itself acts as a pointer to the first element of the array and also if a pointer variable stores the base address of an array then we can manipulate all the array elements using the pointer variable only. Pointers can be associated with the multidimensional arrays (2-D and 3-D arrays) as well.

6.5 EXPLAIN POINTERS TO POINTERS

Double pointer or pointer to pointer is a variable that holds the address of another pointer.

Following is the declaration for a pointer to a pointer

```
datatype ** pointer_name;
```

For example, `int **p;` p is a pointer to pointer

UNDERSTAND HOW TO APPLY OBJECT ORIENTED C++ PROGRAMMING TO DATABASE DEVELOPMENT

7.1 DEFINE DATABASE

DATABASE: is an organized collection of structured information, or data, typically stored electronically in a computer system. A database is usually controlled by a database management system (DBMS).

7.2 DEFINE RELATIONAL DATABASE:

A RELATIONAL DATABASE: is a type of **database that stores and provides access to data points that are related to one another**. Relational databases are based on the relational model, an intuitive, straightforward way of representing data in tables. In a relational database, each row in the table is a record with a unique ID called the key.

7.3 EXPLAIN STEPS OF DATABASE DESIGN

Are you looking to design a database? Whether it's for your workplace or home project, learning how to design a database is a valuable skill. Let's take a look at six easy steps on how to design a database.

➤ **What Is Your Purpose?**

The first place to start when learning how to design a database is to **work out what the purpose is**. Why are you designing a database? No, this isn't just because someone told you do, or because your software needs it. What is the aim of the database? What is it storing?

Thinking about the aim of the database for the first step will help you make decisions about it later on.

➤ **List What You're Storing Data About**

Alright, so you've got your purpose. Now it's time to decide what it's going to store. This is where we go into a little more detail on the database. To give an example, let's say we're creating a database to store customer transactions on your website that sells books.

➤ **Define the Attributes**

Now we have our list of items, or tables, that we're storing data about. The next step is to decide what to store about each of these.

Think about what you need to know about each of these items. What would people want to know? What do you want to know? What represents these items?

➤ **Develop the Relationships**

We have our tables with our columns, or attributes. At the moment, they are just tables, sitting by themselves. To make them work, we need to link them together.

This is what a relationship is. No, I'm not talking about dating; I'm talking about a link between two tables in a database!

A relationship is a **link between two related tables in a database that defines how the data is structured**. Mostly, they represent real-world scenarios.

➤ **Decide On a Unique Identifier**

Each table in a database needs to have a unique identifier. This is a column that can be used to identify this row and *only* this row. This is often called a primary key. It's done as part of the normalisation process of a database.

➤ **Include the Identifiers in Each Table**

Now, we need to go back to our relationships, and learn how to link them. They are linked using these new ID fields we created, and these ID fields need to be in both tables.